

Glossário de Termos — Aula 03

Ambiente de Desenvolvimento e Primeiro Programa Java

Este glossário cobre os termos da Aula 03, onde saímos do papel e entramos no computador. Aqui estão os termos sobre as ferramentas de desenvolvimento e a estrutura básica de um programa Java.

Ferramentas de Desenvolvimento

JDK (Java Development Kit)

Kit de desenvolvimento Java. Contém tudo que é necessário para criar programas: compilador, bibliotecas e a máquina virtual. É a primeira coisa a instalar.

Analogia: É a caixa de ferramentas do programador Java. Sem ela, não dá para construir nada.

JRE (Java Runtime Environment)

Ambiente de execução Java. Permite apenas RODAR programas já prontos, mas não criar novos. O JDK já inclui o JRE.

Analogia: Se o JDK é a fábrica de carros (constrói e anda), o JRE é apenas a estrada (só anda).

JVM (Java Virtual Machine)

Máquina Virtual Java. É o programa que executa o código Java. Existe JVM para Windows, Mac e Linux, por isso Java roda em qualquer sistema.

Analogia: É como um tradutor universal: o código Java é o mesmo, mas a JVM traduz para cada sistema operacional entender.

IDE (Integrated Development Environment)

Ambiente Integrado de Desenvolvimento. Programa que reúne editor de código, compilador, depurador e outras ferramentas num só lugar. Exemplos: IntelliJ IDEA, Eclipse, VS Code.

Analogia: Se o JDK é a caixa de ferramentas, a IDE é a oficina completa: bancada, iluminação, organização. Facilita muito o trabalho.

IntelliJ IDEA

IDE muito popular para desenvolvimento Java. A versão Community é gratuita e suficiente para o curso.

LTS (Long Term Support)

Versão com Suporte de Longo Prazo. O JDK 17 é LTS, ou seja, recebe atualizações de segurança por muitos anos. É a versão recomendada para estudar e trabalhar.

Estrutura de um Programa Java

Classe (class)

Em Java, todo código precisa estar dentro de uma classe. A classe é como um 'contêiner' que organiza o código. O nome da classe deve ser igual ao nome do arquivo.

Analogia: A classe é como uma pasta de arquivo: todo documento (código) precisa estar dentro de uma pasta.

```
public class MeuPrograma { // código aqui dentro }
```

Método main

O ponto de entrada do programa. Quando você executa um programa Java, o computador procura o método main e começa a executar a partir dele.

Analogia: É a porta principal de uma casa: todo visitante (execução) entra por ela.

```
public static void main(String[] args) { // código começa aqui }
```

public

Palavra-chave que significa 'público' — acessível de qualquer lugar. A classe principal e o main precisam ser public.

static

Palavra-chave que significa que o método pertence à classe em si, não a um objeto específico. O main precisa ser static para funcionar.

Dica: Por enquanto, aceite que o main precisa de 'public static void'. O significado ficará mais claro em aulas futuras.

void

Palavra-chave que significa 'vazio' — indica que o método não retorna nenhum valor. O main é void porque ele apenas executa ações.

String[] args

Parâmetro do método main que permite receber argumentos pela linha de comando. Por enquanto, não usaremos, mas é obrigatório na declaração.

System.out.println()

Comando que imprime texto na tela e pula para a próxima linha. É o comando mais básico para mostrar informações ao usuário.

Analogia: É o 'ESCREVA' do pseudocódigo, agora em Java.

```
System.out.println("Olá, mundo!");
```

System.out.print()

Igual ao println, mas NÃO pula linha no final. O próximo texto aparece na mesma linha.

```
System.out.print("Olá "); System.out.print("mundo!"); // Resultado: Olá mundo!
```

Compilação e Execução

Código-fonte

O texto que o programador escreve. Em Java, é salvo em arquivos com extensão .java. É legível por humanos.

Analogia: É a receita escrita à mão: você entende, mas o forno (computador) não.

Compilar / Compilação

Processo de traduzir o código-fonte (.java) para bytecode (.class) — uma linguagem que a JVM entende. Feito pelo compilador javac.

Analogia: É como traduzir a receita do português para a linguagem que o forno robótico entende.

```
javac MeuPrograma.java
```

Bytecode

Código intermediário gerado pela compilação. Não é legível por humanos. A JVM lê e executa o bytecode. Salvo em arquivos .class.

Executar / Execução

Rodar o programa compilado. A JVM interpreta o bytecode e realiza as ações definidas no código.

```
java MeuPrograma
```

Ciclo de desenvolvimento

O processo repetitivo de: 1) Escrever código (.java), 2) Compilar (javac), 3) Executar (java), 4) Testar, 5) Corrigir. Na IDE, os passos 2 e 3 acontecem ao clicar em Run.

Erro de compilação

Erro encontrado durante a compilação. O programa NÃO é gerado. Causas comuns: esquecer ponto e vírgula, nome de variável errado, parênteses sem fechar.

Dica: Leia a mensagem de erro com calma: ela diz o número da linha e o tipo do problema.

Erro de execução (runtime error)

Erro que ocorre quando o programa já está rodando. Exemplo: dividir por zero, acessar posição inválida. O programa compila, mas trava ao executar.

Sintaxe Básica

Ponto e vírgula (;)

Em Java, toda instrução termina com ponto e vírgula. Esquecer dele causa erro de compilação.

Analogia: É como o ponto final de uma frase: indica que aquela instrução acabou.

Chaves { }

Delimitam blocos de código. Tudo entre { e } pertence ao mesmo bloco (classe, método, etc.).

Analogia: São como as paredes de um cômodo: definem onde começa e termina cada espaço.

Comentário

Texto no código que é ignorado pelo compilador. Serve para explicar o que o código faz. Essencial para trabalho em equipe.

```
// Isto é um comentário de uma linha /* Isto é um comentário de múltiplas linhas */
```

Case sensitive

Java diferencia maiúsculas de minúsculas. 'Nome', 'nome' e 'NOME' são três coisas diferentes!

Dica: System.out.println funciona. system.out.println NÃO funciona.